

A SMALL THEME FOR MODERN SLIDES

Views, Security & QWeb

Business UX, ACLs, data loading, and PDF reports

Weblearns Academy
weblearns@training.local
Odoo Full-Stack Developer Program
Lecture materials / July 2026



Agenda

- 1 List, form, search, and kanban patterns
- 2 Groups, access rights, and record rules
- 3 XML/CSV data and view inheritance
- 4 QWeb reports and automation hooks

TOC

1 A theme for modern talks

How this deck is taught

AWESOME-STYLE FRAME

Number rails, angled title art, and section slides are built in.

METROPOLIS DEFAULTS

Progress bars, block styles, and fonts are set from theme options.

DENSE TEACHING

Two-column frames pack objectives, examples, and pitfalls.

Views

Security

XML

QWeb

1 VS01 – VS05

Views, Security & QWeb · teaching block 1

VS01 Intro to Odoo Views and XML Data Loading

Objectives, key points, and lab target

Objectives

- Explain how view records are stored and loaded.
- Choose reliable external ids for view and data records.
- Describe the relationship between model fields and UI architecture.

Key points

- A view is an `ir.ui.view` record with arch XML.
- External ids are part of the module API.
- Data files load in manifest order.
- Views should represent workflows, not database tables only.
- Developer mode is essential for inspecting final architecture.

Lab target

Create a model action, menu, tree view, and form view for a small `training.course` model, then update the module twice to verify idempotency.

VS01 Intro to Odoo Views and XML Data Loading

Teaching notes

Intro to Odoo Views and XML Data Loading is best understood as part of the contract between the ORM, the web client, and the business user. Odoo views are not templates copied into the browser; they are records in `ir.ui.view` that the server combines, validates, and sends to the web client. A senior Odoo developer reads a view or security file as executable configuration: it changes what users can see, which records they can touch, and which operations become safe or...

The design decision starts before writing XML. Start from the model and workflow, then select the smallest view types needed to support the workflow instead of creating every possible view. Good modules keep layout, data defaults, and authorization close to the business process they serve, while avoiding clever view code that hides a weak model design.

Implementation should be small, explicit, and easy to inherit. Declare XML under a data file, reference existing models through `ref`, and keep view ids stable because other modules and upgrades depend on them. Prefer stable external ids, clear XPath anchors, model fields with meaningful states, and security definitions that can be reviewed by a functional lead.

Also remember

- Verification is not only clicking the happy path as an administrator. Install with demo data disabled and enabled,...
- In production, the cost of a poor intro to odoo views and xml data loading decision appears during upgrades, audits,...

VS01 Intro to Odoo Views and XML Data Loading

Example and pitfalls

Minimal window action and menu

The action defines the model and ordered view modes; the menu only points users to that action.

```
<odoo>
  <record id="training_course_action" model="ir.actions.act_window" >
    <field name="name">Courses</field>
    <field name="res_model">training.course</field>
    <field name="view_mode">tree,form,kanban</field>
    <field name="context">{'search_default_active': 1}</field>
  </record>

  <menuitem id="training_menu_root" name="Training"/>
  <menuitem id="training_menu_courses"
# ...
```

Common mistakes

- Changing external ids after another module inherits them.
- Loading views before the model fields exist.
- Putting business rules only in XML instead of model methods.
- Forgetting that manifest data file order is significant.

VS01 Intro to Odoo Views and XML Data Loading

Extra patterns

View loading checkpoints

-

VS02 List Views, Sorting, and Limits

Objectives, key points, and lab target

Objectives

- Build a readable list view for operational records.
- Use `default_order` and `limit` intentionally.
- Select list columns that support the primary user decision.

Key points

- Tree views are now commonly referred to as list views in newer Odoo discussions.
- `default_order` belongs on the view arch.
- `limit` changes the initial row count for that view.
- Columns should reflect the user decision.
- Decorations and modifiers may require invisible helper fields.

Lab target

Replace a wide list view with a decision-focused list, add `default_order`, and compare the generated read fields in the browser network tab.

VS02 List Views, Sorting, and Limits

Teaching notes

List Views, Sorting, and Limits is best understood as part of the contract between the ORM, the web client, and the business user. The list view is usually the first screen where users scan volume, compare records, and decide what needs attention. A senior Odoo developer reads a view or security file as executable configuration: it changes what users can see, which records they can touch, and which operations become safe or dangerous in production.

The design decision starts before writing XML. Choose columns by asking what a user must know before opening a form; do not mirror every field in the model. Good modules keep layout, data defaults, and authorization close to the business process they serve, while avoiding clever view code that hides a weak model design.

Implementation should be small, explicit, and easy to inherit. Use `default_order` on indexed or naturally sortable fields and reserve limit for dashboards or queues where a short list is part of the design. Prefer stable external ids, clear XPath anchors, model fields with meaningful states, and security definitions that can be reviewed by a functional lead.

Also remember

- Verification is not only clicking the happy path as an administrator. Check list load time with realistic data,...
- In production, the cost of a poor list views, sorting, and limits decision appears during upgrades, audits, and...

VS02 List Views, Sorting, and Limits

Example and pitfalls

Operational list with stable ordering

The list opens with recent courses first and only includes fields needed to triage the course catalog.

```
<record id="training_course_view_tree" model="ir.ui.view">
  <field name="name">training.course.tree</field>
  <field name="model">training.course</field>
  <field name="arch" type="xml">
    <tree string="Courses" default_order="start_date">
      <field name="name"/>
      <field name="teacher_id"/>
      <field name="start_date"/>
      <field name="seat_count"/>
      <field name="state"/>
    </tree>
  </field>
</record>
```

...

Common mistakes

- Using a non-indexed computed field as the default sort on a large table.
- Adding every field because it is available.
- Using limit to hide bad performance rather than fixing the cause.
- Forgetting multi-company or active filters when reviewing list content.

VS03 Editable Lists, Mass Editing, and the Handle Widget

Objectives, key points, and lab target

Objectives

- Enable safe inline edits for simple records.
- Understand when mass editing is appropriate.
- Use handle widgets for sequence-based ordering.

Key points

- Inline editing is a workflow choice, not a default.
- `multi_edit` applies one value to many selected rows.
- The handle widget requires an orderable sequence field.
- Server constraints remain mandatory.
- Editable lists are risky on approval records.

Lab target

Add an editable list to course session lines, include a handle sequence, and prove that a negative duration is blocked by a Python constraint.

VS03 Editable Lists, Mass Editing, and the Handle Widget

Teaching notes

Editable Lists, Mass Editing, and the Handle Widget is best understood as part of the contract between the ORM, the web client, and the business user. Editable lists reduce navigation when users are maintaining line-like records, but they also reduce the friction that normally makes users review a form. A senior Odoo developer reads a view or security file as executable configuration: it changes what users can see, which records they can touch, and which operations become...

The design decision starts before writing XML. Only make fields editable when validation is local and obvious; complex onchange chains, attachments, chatter, or approvals usually deserve a form. Good modules keep layout, data defaults, and authorization close to the business process they serve, while avoiding clever view code that hides a weak model design.

Implementation should be small, explicit, and easy to inherit. Use `editable='bottom'` for controlled creation, `multi_edit='1'` for deliberate mass changes, and `widget='handle'` with an integer sequence field for manual ordering. Prefer stable external ids, clear XPath anchors, model fields with meaningful states, and security definitions that can be reviewed by a functional lead.

Also remember

- Verification is not only clicking the happy path as an administrator. Test create, edit, discard, required fields,...
- In production, the cost of a poor editable lists, mass editing, and the handle widget decision appears during...

VS03 Editable Lists, Mass Editing, and the Handle Widget

Example and pitfalls

Editable line list with sequence handle

Users can reorder and maintain simple training lines without opening each form, while required fields remain enforced.

```
<tree editable="bottom" multi_edit="1" default_order="1"
  <field name="sequence" widget="handle"/>
  <field name="name" required="1"/>
  <field name="duration_hours"/>
  <field name="price"/>
</tree>
```

Common mistakes

- Enabling editable lists on records with complex workflow transitions.
- Forgetting to include sequence in `_order` on the model.
- Assuming mass edit bypasses access rights.
- Relying on `onchange` instead of constraints for critical validation.

VS04 List Decorations, Field Decorations, Widths, and Hidden Labels

Objectives, key points, and lab target

Objectives

- Apply visual decorations without changing data semantics.
- Use invisible fields as decoration inputs.
- Tune field width and labels for dense operational screens.

Key points

- Row decorations use expressions evaluated by the client.
- Field decorations work well on status or KPI fields.
- Invisible helper fields can feed modifiers and decorations.
- Width hints are guidance, not a security feature.
- Do not encode business rules only as colors.

Lab target

Create three sample records that trigger danger, warning, and success decorations, then ask another user to explain the meaning without reading the XML.

VS04 List Decorations, Field Decorations, Widths, and Hidden Labels

Teaching notes

List Decorations, Field Decorations, Widths, and Hidden Labels is best understood as part of the contract between the ORM, the web client, and the business user. Decorations are a fast way to turn a list into an exception queue, but they should reinforce a business meaning already present in the data. A senior Odoo developer reads a view or security file as executable configuration: it changes what users can see, which records they can touch, and which operations become...

The design decision starts before writing XML. Design the visual language first: one color for danger, one for success, and a small number of badges so users do not learn to ignore noise. Good modules keep layout, data defaults, and authorization close to the business process they serve, while avoiding clever view code that hides a weak model design.

Implementation should be small, explicit, and easy to inherit. Use row decoration attributes, field-level decorations, optional fields, width hints, and nolabel where they clarify the screen without hiding important data. Prefer stable external ids, clear XPath anchors, model fields with meaningful states, and security definitions that can be reviewed by a functional lead.

Also remember

- Verification is not only clicking the happy path as an administrator. Review decorations with sample records in every...
- In production, the cost of a poor list decorations, field decorations, widths, and hidden labels decision appears...

VS04 List Decorations, Field Decorations, Widths, and Hidden Labels

Example and pitfalls

Exception-oriented decorated list

The list highlights cancellation, under-filled courses, and completion while keeping the comparison field available invisibly.

```
<tree decoration-danger="state == 'cancel'"
      decoration-warning="seat_count < minimum_seats"
      decoration-success="state == 'done'">
  <field name="name" width="240px"/>
  <field name="minimum_seats" invisible="1"/>
  <field name="seat_count" decoration-warning="seat_count < minimum_seats"/>
  <field name="state" widget="badge"/>
</tree>
```

Common mistakes

- Using many colors for unrelated meanings.
- Forgetting XML escaping for comparison operators.
- Treating invisible fields as secure.
- Making labels disappear when the field meaning is not obvious.

VS05 List Totals, Averages, and Grouped Lists

Objectives, key points, and lab target

Objectives

- Use sum and avg footers on numeric fields.
- Understand how grouped list aggregations help operations.
- Avoid misleading totals on filtered data.

Key points

- sum and avg attributes create list footers.
- Grouped lists aggregate per group and overall.
- Stored fields are safer for SQL aggregation.
- Currency and unit of measure context matters.
- Totals reflect the current domain and access rules.

Lab target

Add sum and average footers to a list, group by teacher, and compare grouped totals against a manual `read_group` call in the Odoo shell.

VS05 List Totals, Averages, and Grouped Lists

Teaching notes

List Totals, Averages, and Grouped Lists is best understood as part of the contract between the ORM, the web client, and the business user. Totals and averages make a list view useful for reconciliation, capacity planning, and quick financial checks. A senior Odoo developer reads a view or security file as executable configuration: it changes what users can see, which records they can touch, and which operations become safe or dangerous in production.

The design decision starts before writing XML. Decide whether a metric should be summed, averaged, counted, or left out; not every numeric field communicates meaningful aggregate information. Good modules keep layout, data defaults, and authorization close to the business process they serve, while avoiding clever view code that hides a weak model design.

Implementation should be small, explicit, and easy to inherit. Add sum and avg labels directly on fields, combine with `groupby` where appropriate, and ensure computed fields are stored if aggregation must happen in SQL. Prefer stable external ids, clear XPath anchors, model fields with meaningful states, and security definitions that can be reviewed by a functional lead.

Also remember

- Verification is not only clicking the happy path as an administrator. Validate totals under filters, groups,...
- In production, the cost of a poor list totals, averages, and grouped lists decision appears during upgrades, audits,...

VS05 List Totals, Averages, and Grouped Lists

Example and pitfalls

Capacity and revenue footers

The footer gives immediate operational totals without requiring an export to a spreadsheet.

```
<tree default_order="start_date desc">
  <field name="name"/>
  <field name="teacher_id"/>
  <field name="seat_count" sum="Total Seats"/>
  <field name="price_total" sum="Total Revenue"/>
  <field name="satisfaction_score" avg="Average Score"/>
</tree>
```

Common mistakes

- Summing percentages or scores that should be averaged.
- Aggregating non-stored computed fields that cannot be grouped efficiently.
- Ignoring currency mixing across companies.
- Letting totals imply a complete population when filters are active.

2 VS06 – VS10

Views, Security & QWeb · teaching block 2

VS06 Search Views, Default Filters, and Dynamic Date Filters

Objectives, key points, and lab target

Objectives

- Build search filters that match daily work.
- Set default filters from actions.
- Use dynamic datetime domains safely.

Key points

- Search filters are reusable domain snippets.
- `search_default_filter_name` activates a named filter.
- Dynamic dates are evaluated relative to context.
- Search fields should map to indexed or useful fields.
- Menus can share a search view with different contexts.

Lab target

Create a menu that opens courses with Active and Starts This Month enabled by default, then add a second menu that opens the same model without those defaults.

VS06 Search Views, Default Filters, and Dynamic Date Filters

Teaching notes

Search Views, Default Filters, and Dynamic Date Filters is best understood as part of the contract between the ORM, the web client, and the business user. A search view turns a generic model list into a work queue, a reporting entry point, or a compliance review screen. A senior Odoo developer reads a view or security file as executable configuration: it changes what users can see, which records they can touch, and which operations become safe or dangerous in production.

The design decision starts before writing XML. Start with the questions users ask every day, then encode those questions as filters, separators, date filters, and search fields with understandable labels. Good modules keep layout, data defaults, and authorization close to the business process they serve, while avoiding clever view code that hides a weak model design.

Implementation should be small, explicit, and easy to inherit. Declare filters in the search view, pass `search_default` keys from the action context, and use `context_today` or relative datetime helpers carefully in domains. Prefer stable external ids, clear XPath anchors, model fields with meaningful states, and security definitions that can be reviewed by a functional lead.

Also remember

- Verification is not only clicking the happy path as an administrator. Verify defaults from every menu and action that...
- In production, the cost of a poor search views, default filters, and dynamic date filters decision appears during...

VS06 Search Views, Default Filters, and Dynamic Date Filters

Example and pitfalls

Default active and this-month filters

The search view offers direct text search, a reusable active filter, a dynamic date filter, and a group-by shortcut.

```
<search string="Courses">
  <field name="name"/>
  <field name="teacher_id"/>
  <filter name="active" string="Active" domain="[('active', '=', True)]"/>
  <filter name="this_month" string="Starts This Month"
    domain="[('start_date', '>=', context_today().strftime('%Y-%m-01'))]"/>
  <filter name="group_teacher" string="Teacher" context="{ 'group_by': 'teacher_id' }"/>
</search>
```

Common mistakes

- Using dynamic filters for legally fixed reporting periods.
- Forgetting that action context controls default filters.
- Adding filters users cannot explain.
- Creating domains that ignore timezone expectations.

VS07 Search Panel and Groupby UX

Objectives, key points, and lab target

Objectives

- Use search panels for guided filtering.
- Design groupby options that explain the data.
- Avoid filter combinations that create empty screens.

Key points

- Search panel fields guide filtering from the left side.
- Groupby filters change result organization, not just filtering.
- Counters can have performance cost.
- Hierarchies need clean parent relationships.
- Use labels that match business language.

Lab target

Add a search panel to a course model with categories and teachers, then measure the difference between counters enabled and disabled on sample data.

VS07 Search Panel and Groupby UX

Teaching notes

Search Panel and Groupby UX is best understood as part of the contract between the ORM, the web client, and the business user. The search panel is a navigation aid for users who think in categories more than domains. A senior Odoo developer reads a view or security file as executable configuration: it changes what users can see, which records they can touch, and which operations become safe or dangerous in production.

The design decision starts before writing XML. Use it when the model has meaningful many2one, many2many, selection, or hierarchy fields that users naturally use to narrow a dataset. Good modules keep layout, data defaults, and authorization close to the business process they serve, while avoiding clever view code that hides a weak model design.

Implementation should be small, explicit, and easy to inherit. Define searchpanel fields separately from groupby filters and decide whether a user needs counters, multi-select, or category dependencies. Prefer stable external ids, clear XPath anchors, model fields with meaningful states, and security definitions that can be reviewed by a functional lead.

Also remember

- Verification is not only clicking the happy path as an administrator. Test large datasets because counters and many...
- In production, the cost of a poor search panel and groupby ux decision appears during upgrades, audits, and support...

VS07 Search Panel and Groupby UX

Example and pitfalls

Category and teacher search panel

The panel supports guided category and teacher filtering while groupby remains a separate analytical option.

```
<search string="Courses">
  <field name="name"/>
  <searchpanel>
    <field name="category_id" icon="fa-folder-open"/>
    <field name="teacher_id" select="multi" icon="fa-user"/>
  </searchpanel>
  <filter name="group_category" string="Category" context="{ 'group_by': 'category_id' }"/>
</search>
```

Common mistakes

- Enabling counters on a model with slow domains without measuring.
- Duplicating every groupby as a search panel field.
- Using technical field names as labels.
- Assuming search panel visibility is a security boundary.

VS08 Form Customization with Groups, Tags, and Widgets

Objectives, key points, and lab target

Objectives

- Create clean form layouts with semantic grouping.
- Use tags and widgets where they improve comprehension.
- Keep forms aligned with business workflow.

Key points

- Forms should tell a story about one record.
- sheet and group provide standard Odoo visual structure.
- many2many_tags is compact but can hide detail.
- Place technical fields only when they help the workflow.
- Readable anchors make later inheritance safer.

Lab target

Redesign a flat form into two groups and one tag field, then inspect the final architecture after installing a second module that inherits it.

VS08 Form Customization with Groups, Tags, and Widgets

Teaching notes

Form Customization with Groups, Tags, and Widgets is best understood as part of the contract between the ORM, the web client, and the business user. A form view is where the user understands and changes one business object, so layout communicates priority as much as labels do. A senior Odoo developer reads a view or security file as executable configuration: it changes what users can see, which records they can touch, and which operations become safe or dangerous in...

The design decision starts before writing XML. Group fields by decision area, place identity fields first, and use widgets such as `many2many_tags` only when the compact representation is easier to read. Good modules keep layout, data defaults, and authorization close to the business process they serve, while avoiding clever view code that hides a weak model design.

Implementation should be small, explicit, and easy to inherit. Use sheet, group, separator, field options, placeholder, widget, and help text in a way that supports scanning without turning the form into a dashboard. Prefer stable external ids, clear XPath anchors, model fields with meaningful states, and security definitions that can be reviewed by a functional lead.

Also remember

- Verification is not only clicking the happy path as an administrator. Review the form at common screen sizes, with...
- In production, the cost of a poor form customization with groups, tags, and widgets decision appears during upgrades,...

VS08 Form Customization with Groups, Tags, and Widgets

Example and pitfalls

Structured course form

The form separates identity from delivery so users can understand the record before editing schedule details.

```
<form string="Course">
  <sheet>
    <group>
      <group string="Identity">
        <field name="name" placeholder="Functional Odoo Basics"/>
        <field name="category_id"/>
        <field name="tag_ids" widget="many2many_tags"/>
      </group>
      <group string="Delivery">
        <field name="teacher_id"/>
      </group>
    </group>
  </sheet>
</form>
```

Common mistakes

- Making one enormous group with no visual hierarchy.
- Using tag widgets for records where users need columns or attributes.
- Hiding required context in tabs users rarely open.
- Letting inherited views depend on brittle positional anchors.

VS09 Notebook Pages, Button Containers, Header Buttons, and Statusbars

Objectives, key points, and lab target

Objectives

- Use notebook pages for secondary information.
- Place workflow buttons in the header.
- Represent state with a statusbar widget.

Key points

- Header buttons should map to business transitions.
- statusbar displays the lifecycle field.
- Notebook pages are for secondary or repeated information.
- Button visibility should match server-side rules.
- Use `button_box` for smart buttons, not random actions.

Lab target

Add a draft-confirmed-done workflow with header buttons, then test every transition as a manager and as a read-only user.

VS09 Notebook Pages, Button Containers, Header Buttons, and Statusbars

Teaching notes

Notebook Pages, Button Containers, Header Buttons, and Statusbars is best understood as part of the contract between the ORM, the web client, and the business user. The header and notebook areas turn a simple form into a workflow screen without requiring custom JavaScript. A senior Odoo developer reads a view or security file as executable configuration: it changes what users can see, which records they can touch, and which operations become safe or dangerous in production.

The design decision starts before writing XML. Put state-changing actions where users expect workflow controls, and use notebook pages for details that matter after the primary identity is understood. Good modules keep layout, data defaults, and authorization close to the business process they serve, while avoiding clever view code that hides a weak model design.

Implementation should be small, explicit, and easy to inherit. Implement object buttons for Python methods, action buttons for client actions or window actions, and `statusbar_visible` to show the intended lifecycle. Prefer stable external ids, clear XPath anchors, model fields with meaningful states, and security definitions that can be reviewed by a functional lead.

Also remember

- Verification is not only clicking the happy path as an administrator. Verify every button in every state with access...
- In production, the cost of a poor notebook pages, button containers, header buttons, and statusbars decision appears...

VS09 Notebook Pages, Button Containers, Header Buttons, and Statusbars

Example and pitfalls

Workflow header and notebook

The header owns the lifecycle while the notebook separates operational details from notes.

```
<form string="Course">
  <header>
    <button name="action_confirm" type="object" st
      class="oe_highlight" invisible="state
    <button name="action_done" type="object" string
      invisible="state != 'confirmed'"/>
    <field name="state" widget="statusbar"
      statusbar_visible="draft,confirmed,done"/>
  </header>
</sheet>
# ...
```

Common mistakes

- Putting workflow buttons in random groups inside the form.
- Showing states in the statusbar that users cannot reach.
- Hiding buttons only in XML while Python methods remain too permissive.
- Using notebooks for required first-step data.

VS10 Attrs Migration and Dynamic Readonly, Invisible, Required Rules

Objectives, key points, and lab target

Objectives

- Convert legacy attrs and states into modern modifier expressions.
- Use readonly, invisible, and required conditions safely.
- Design dynamic statusbar behavior without hiding business truth.

Key points

- Modern modifiers use direct expressions such as `invisible="state != 'draft'"`.
- Fields referenced by expressions must be available to the client.
- Required conditions should match model constraints where possible.
- Migration is a refactoring opportunity.
- Server-side validation remains authoritative.

Lab target

Take a legacy form using attrs, convert three modifiers to direct expressions, and run through draft, confirmed, done, and cancel examples.

VS10 Attrs Migration and Dynamic Readonly, Invisible, Required Rules

Teaching notes

Attrs Migration and Dynamic Readonly, Invisible, Required Rules is best understood as part of the contract between the ORM, the web client, and the business user. Odoo's view modifiers evolved from attrs dictionaries toward direct Python-like expressions in XML attributes. A senior Odoo developer reads a view or security file as executable configuration: it changes what users can see, which records they can touch, and which operations become safe or dangerous in production.

The design decision starts before writing XML. Treat migration as a chance to simplify conditions rather than translating every old attrs expression mechanically. Good modules keep layout, data defaults, and authorization close to the business process they serve, while avoiding clever view code that hides a weak model design.

Implementation should be small, explicit, and easy to inherit. Write direct invisible, readonly, and required expressions against fields loaded in the view, and keep dynamic `statusbar_visible` decisions tied to meaningful workflow states. Prefer stable external ids, clear XPath anchors, model fields with meaningful states, and security definitions that can be reviewed by a functional lead.

Also remember

- Verification is not only clicking the happy path as an administrator. Compare the final architecture before and after...
- In production, the cost of a poor attrs migration and dynamic readonly, invisible, required rules decision appears...

VS10 Attrs Migration and Dynamic Readonly, Invisible, Required Rules

Example and pitfalls

Modern modifiers replacing attrs

The conditions are readable in place and no longer require a nested attrs dictionary.

```
<field name="cancel_reason"
      invisible="state != 'cancel'"
      required="state == 'cancel'"
      readonly="state in ('done', 'cancel')"/>
<field name="state" widget="statusbar"
      statusbar_visible="draft,confirmed,done,cancel"/>
```

Common mistakes

- Migrating syntax without checking behavior in every state.
- Referencing a field not present in the view.
- Using invisible required fields without a server default.
- Relying on UI required rules instead of model constraints.

3 VS11 – VS15

Views, Security & QWeb · teaching block 3

VS11 Buttons that Open Forms, URL Actions, and Disabled Actions

Objectives, key points, and lab target

Objectives

- Return window actions from object buttons.
- Create URL actions for external systems.
- Disable buttons through state and group rules.

Key points

- type='object' calls a model method.
- Window actions can open a specific record form.
- URL actions should be generated from trusted values.
- Button visibility is not authorization.
- Use groups on buttons for role-specific actions.

Lab target

Add a smart button that opens the teacher partner and a separate URL action to documentation, then secure the object method with a group check.

VS11 Buttons that Open Forms, URL Actions, and Disabled Actions

Teaching notes

Buttons that Open Forms, URL Actions, and Disabled Actions is best understood as part of the contract between the ORM, the web client, and the business user. Buttons connect views to behavior, so every button should make the next user step obvious and safe. A senior Odoo developer reads a view or security file as executable configuration: it changes what users can see, which records they can touch, and which operations become safe or dangerous in production.

The design decision starts before writing XML. Choose object buttons for model methods, action buttons for existing actions, stat buttons for counters, and URL actions only when the target is trustworthy. Good modules keep layout, data defaults, and authorization close to the business process they serve, while avoiding clever view code that hides a weak model design.

Implementation should be small, explicit, and easy to inherit. Return `ir.actions.act_window` dictionaries for opening forms, use `target='new'` for modal flows, and make visibility conditions mirror server-side checks. Prefer stable external ids, clear XPath anchors, model fields with meaningful states, and security definitions that can be reviewed by a functional lead.

Also remember

- Verification is not only clicking the happy path as an administrator. Test buttons with missing related records,...
- In production, the cost of a poor buttons that open forms, url actions, and disabled actions decision appears during...

VS11 Buttons that Open Forms, URL Actions, and Disabled Actions

Example and pitfalls

Open the related partner form

The object button returns a precise form action instead of relying on a generic menu action.

```
def action_open_teacher_partner(self):
    self.ensure_one()
    return {
        "type": "ir.actions.act_window",
        "name": "Teacher",
        "res_model": "res.partner",
        "res_id": self.teacher_id.id,
        "view_mode": "form",
        "target": "current",
    }
```

Common mistakes

- Building URLs from user input without validation.
- Hiding a button but leaving the method callable by any user.
- Returning an action without `res_id` when a specific form is expected.
- Using buttons for operations that should be computed automatically.

VS12 Kanban Views, Templates, and Images

Objectives, key points, and lab target

Objectives

- Build kanban cards with QWeb templates.
- Render image fields efficiently.
- Choose card content that supports quick triage.

Key points

- Kanban templates use QWeb under the kanban arch.
- Fields must be declared for use in templates.
- Images should have fallbacks.
- Cards should prioritize the next action.
- Keep templates easy to inherit.

Lab target

Create a kanban card with an image, title, teacher, and state badge, then test it with records that do and do not have images.

VS12 Kanban Views, Templates, and Images

Teaching notes

Kanban Views, Templates, and Images is best understood as part of the contract between the ORM, the web client, and the business user. Kanban views are visual workspaces for records that benefit from cards, images, statuses, or stage-based scanning. A senior Odoo developer reads a view or security file as executable configuration: it changes what users can see, which records they can touch, and which operations become safe or dangerous in production.

The design decision starts before writing XML. Design the card around the action users take next, not around showing every field in a compact rectangle. Good modules keep layout, data defaults, and authorization close to the business process they serve, while avoiding clever view code that hides a weak model design.

Implementation should be small, explicit, and easy to inherit. Declare fields before templates, use `kanban_image` or `image_data` URLs carefully, and keep card templates simple enough for inheritance. Prefer stable external ids, clear XPath anchors, model fields with meaningful states, and security definitions that can be reviewed by a functional lead.

Also remember

- Verification is not only clicking the happy path as an administrator. Verify empty images, slow networks, mobile...
- In production, the cost of a poor kanban views, templates, and images decision appears during upgrades, audits, and...

VS12 Kanban Views, Templates, and Images

Example and pitfalls

Course kanban with image fallback

The card declares fields used by the QWeb template and uses the standard kanban image helper.

```
<kanban>
  <field name="name"/>
  <field name="teacher_id"/>
  <field name="image_128"/>
  <templates>
    <t t-name="kanban-box">
      <div class="oe_kanban_global_click o_kanban_record">
        
        <strong><field name="name"/></strong>
      </div>
    </t>
  </templates>
# ...
```

Common mistakes

- Using undeclared fields inside the kanban template.
- Putting large binary images directly into crowded cards.
- Making every element clickable with unclear affordance.
- Forgetting fallback behavior when images are empty.

VS13 Kanban Drag and Drop, Handle Ordering, and Progressbars

Objectives, key points, and lab target

Objectives

- Enable stage-based drag and drop in kanban.
- Use sequence fields for manual ordering.
- Display progressbar summaries on kanban columns.

Key points

- `default_group_by` creates a column-oriented kanban.
- Drag and drop writes the grouped field.
- Handle ordering needs a sequence field.
- Progressbars summarize grouped records.
- Server constraints must protect invalid moves.

Lab target

Build a stage kanban, add a constraint that blocks moving done records back to draft, and verify the drag is rejected.

VS13 Kanban Drag and Drop, Handle Ordering, and Progressbars

Teaching notes

Kanban Drag and Drop, Handle Ordering, and Progressbars is best understood as part of the contract between the ORM, the web client, and the business user. Kanban becomes operational when records can move between stages and users can see column-level progress. A senior Odoo developer reads a view or security file as executable configuration: it changes what users can see, which records they can touch, and which operations become safe or dangerous in production.

The design decision starts before writing XML. Only enable drag and drop when a move is a valid business transition; otherwise the UI invites users to bypass workflow thinking. Good modules keep layout, data defaults, and authorization close to the business process they serve, while avoiding clever view code that hides a weak model design.

Implementation should be small, explicit, and easy to inherit. Use `default_group_by` for stages, a handle sequence for ordering, and progressbar definitions for quick aggregated status. Prefer stable external ids, clear XPath anchors, model fields with meaningful states, and security definitions that can be reviewed by a functional lead.

Also remember

- Verification is not only clicking the happy path as an administrator. Test dragging with record rules, state...
- In production, the cost of a poor kanban drag and drop, handle ordering, and progressbars decision appears during...

VS13 Kanban Drag and Drop, Handle Ordering, and Progressbars

Example and pitfalls

Stage kanban with progressbar

The kanban groups by stage, allows drag ordering, and visualizes state distribution per column.

```
<kanban default_group_by="stage_id" records_draggable=
  <field name="stage_id"/>
  <field name="sequence"/>
  <field name="state"/>
  <progressbar field="state"
    colors="{ 'draft': 'muted', 'confirmed': 'warning', 'done': 'success' }"/>
  <templates>
    <t t-name="kanban-box">
      <div class="oe_kanban_global_click">
        <field name="sequence" widget="handle"/>
      </div>
    </t>
  </templates>
# ...
```

Common mistakes

- Allowing drag and drop when state transitions require approvals.
- Forgetting access rules on the grouped field.
- Using progressbar colors with states that do not exist.
- Assuming client drag behavior replaces server validation.

VS14 Graph, Pivot, Cohort, and Gantt Views

Objectives, key points, and lab target

Objectives

- Select the correct analytical view for the question.
- Configure measures and intervals.
- Understand enterprise-only planning views such as cohort and gantt.

Key points

- Graph views visualize aggregated measures.
- Pivot views support exploratory analysis.
- Cohort views track behavior over periods.
- Gantt views schedule records across dates.
- Stored fields make reporting reliable.

Lab target

Create graph and pivot views for courses, seed five months of data, and verify one total manually with read_group.

VS14 Graph, Pivot, Cohort, and Gantt Views

Teaching notes

Graph, Pivot, Cohort, and Gantt Views is best understood as part of the contract between the ORM, the web client, and the business user. Analytical views answer different questions: graph shows shape, pivot supports exploration, cohort tracks retention, and gantt plans time. A senior Odoo developer reads a view or security file as executable configuration: it changes what users can see, which records they can touch, and which operations become safe or dangerous in production.

The design decision starts before writing XML. Start from the business question and expected measure, then choose the view type rather than adding every reporting view automatically. Good modules keep layout, data defaults, and authorization close to the business process they serve, while avoiding clever view code that hides a weak model design.

Implementation should be small, explicit, and easy to inherit. Declare measures, row and column fields, date intervals, and dependencies with stored fields so the database can aggregate predictably. Prefer stable external ids, clear XPath anchors, model fields with meaningful states, and security definitions that can be reviewed by a functional lead.

Also remember

- Verification is not only clicking the happy path as an administrator. Validate analytical views with known fixtures...
- In production, the cost of a poor graph, pivot, cohort, and gantt views decision appears during upgrades, audits, and...

VS14 Graph, Pivot, Cohort, and Gantt Views

Example and pitfalls

Revenue pivot and graph

The pivot supports analysis by teacher and month, while the graph shows capacity trends.

```
<pivot string="Course Revenue">
  <field name="teacher_id" type="row"/>
  <field name="start_date" interval="month" type="column"/>
  <field name="price_total" type="measure"/>
</pivot>

<graph string="Seats by Month" type="bar">
  <field name="start_date" interval="month"/>
  <field name="seat_count" type="measure"/>
</graph>
```

Common mistakes

- Using non-stored computed measures.
- Mixing unrelated measures in one graph.
- Forgetting enterprise availability for cohort and gantt.
- Ignoring access-rule effects on reported totals.

VS15 Security Overview: Six Components, Groups, Categories, and Hierarchy

Objectives, key points, and lab target

Objectives

- Describe the main layers of Odoo security.
- Organize groups under useful application categories.
- Explain parent-child group relationships to functional users.

Key points

- Security has more than CSV access rights.
- Groups collect permissions and UI visibility.
- Categories organize groups in the user form.
- Record rules restrict rows after model access is granted.
- Business methods must still check sensitive transitions.

Lab target

Create three job-role groups for a training app and write a one-sentence business description for each before adding permissions.

VS15 Security Overview: Six Components, Groups, Categories, and Hierarchy

Teaching notes

Security Overview: Six Components, Groups, Categories, and Hierarchy is best understood as part of the contract between the ORM, the web client, and the business user. Odoo security is layered: model access rights, record rules, groups, field/view/action group restrictions, sudo boundaries, and business-method checks all matter. A senior Odoo developer reads a view or security file as executable configuration: it changes what users can see, which records they can touch, and...

The design decision starts before writing XML. Design roles from real job responsibilities, then map them to groups and categories that users can understand in the Settings UI. Good modules keep layout, data defaults, and authorization close to the business process they serve, while avoiding clever view code that hides a weak model design.

Implementation should be small, explicit, and easy to inherit. Create module categories for clean group presentation and define parent-child categories only when they reflect a real application hierarchy. Prefer stable external ids, clear XPath anchors, model fields with meaningful states, and security definitions that can be reviewed by a functional lead.

Also remember

- Verification is not only clicking the happy path as an administrator. Verify each role with a separate user, not by...
- In production, the cost of a poor security overview: six components, groups, categories, and hierarchy decision...

VS15 Security Overview: Six Components, Groups, Categories, and Hierarchy

Example and pitfalls

Application category and groups

The category makes the group visible in the expected application section of the user form.

```
<record id="module_category_training" model="ir.module"  
  <field name="name">Training</field>  
  <field name="sequence">35</field>  
</record>  
  
<record id="group_training_user" model="res.groups">  
  <field name="name">Training User</field>  
  <field name="category_id" ref="module_category_training"/>  
</record>
```

Common mistakes

- Assuming menus are security controls.
- Designing groups from developer convenience instead of job roles.
- Granting broad access and trying to patch it later with record rules.
- Testing security only as the administrator.

4 VS16 – VS20

Views, Security & QWeb · teaching block 4

VS16 Implied Groups and Combobox Group Selection

Objectives, key points, and lab target

Objectives

- Use implied_ids to build role inheritance.
- Understand group selection behavior in the user form.
- Avoid accidental privilege escalation through implied groups.

Key points

- implied_ids grants included groups automatically.
- Role inheritance should go from broad basic access to stronger roles.
- Group categories affect how selections appear on users.
- Removing an implying group can affect implied membership.
- Implication is not a substitute for record rules.

Lab target

Create user, teacher, and manager groups where manager implies teacher and teacher implies user, then inspect the effective groups on a test user.

VS16 Implied Groups and Combobox Group Selection

Teaching notes

Implied Groups and Combobox Group Selection is best understood as part of the contract between the ORM, the web client, and the business user. Implied groups let one role include another role, which keeps permissions maintainable when managers should also have normal user capabilities. A senior Odoo developer reads a view or security file as executable configuration: it changes what users can see, which records they can touch, and which operations become safe or dangerous...

The design decision starts before writing XML. Design implication from smaller to larger roles and make the chain obvious; hidden implication is a common source of unexpected access. Good modules keep layout, data defaults, and authorization close to the business process they serve, while avoiding clever view code that hides a weak model design.

Implementation should be small, explicit, and easy to inherit. Set `implied_ids` with the `many2many` command syntax and use exclusive category behavior carefully when groups are presented as a combobox-style selection. Prefer stable external ids, clear XPath anchors, model fields with meaningful states, and security definitions that can be reviewed by a functional lead.

Also remember

- Verification is not only clicking the happy path as an administrator. Test adding and removing groups from a user...
- In production, the cost of a poor implied groups and combobox group selection decision appears during upgrades,...

VS16 Implied Groups and Combobox Group Selection

Example and pitfalls

Manager implies user

A manager automatically receives the base user permissions instead of duplicating them.

```
<record id="group_training_manager" model="res.groups">
  <field name="name">Training Manager</field>
  <field name="category_id" ref="module_category_training">
  <field name="implied_ids" eval="[(4, ref('training_group'))]">
</record>
```

Common mistakes

- Creating circular or confusing implication chains.
- Using implied groups to hide overly broad access rights.
- Forgetting that implied groups affect all users assigned the parent group.
- Not testing group removal behavior.

VS17 Access Rights in CSV and XML

Objectives, key points, and lab target

Objectives

- Write `ir.model.access.csv` rows correctly.
- Know when XML access records are useful.
- Separate model access from row-level record rules.

Key points

- CSV rows live in `security/ir.model.access.csv`.
- `perm_create/read/write/unlink` are independent booleans.
- The `model_id` value references `model_model_name`.
- Record rules filter rows after access rights pass.
- XML access records are possible but less common.

Lab target

Create an access CSV for course and session models, then confirm a read-only user cannot create from the UI or via RPC.

VS17 Access Rights in CSV and XML

Teaching notes

Access Rights in CSV and XML is best understood as part of the contract between the ORM, the web client, and the business user. Access rights answer whether a group may create, read, write, or delete records of a model at all. A senior Odoo developer reads a view or security file as executable configuration: it changes what users can see, which records they can touch, and which operations become safe or dangerous in production.

The design decision starts before writing XML. Use CSV for normal module permissions because it is compact, reviewable, and recognized by Odoo conventions. Good modules keep layout, data defaults, and authorization close to the business process they serve, while avoiding clever view code that hides a weak model design.

Implementation should be small, explicit, and easy to inherit. Use XML only for exceptional cases where you need update behavior, conditions around noupdate data, or integration with a generated data process. Prefer stable external ids, clear XPath anchors, model fields with meaningful states, and security definitions that can be reviewed by a functional lead.

Also remember

- Verification is not only clicking the happy path as an administrator. Verify access with a user who has only the...
- In production, the cost of a poor access rights in csv and xml decision appears during upgrades, audits, and support...

VS17 Access Rights in CSV and XML

Example and pitfalls

Access CSV for course users

Base users can read courses while managers receive full model-level access.

```
id,name,model_id:id,group_id:id,perm_read,perm_write,perm_unlink,access_training_course_user,training.course user,model_id:id,group_id:id,perm_read,perm_write,perm_unlink,access_training_course_manager,training.course manager
```

Common mistakes

- Using record rules to compensate for missing model access.
- Misspelling model external ids in CSV.
- Granting unlink by default to operational users.
- Forgetting access rows for transient models used by wizards.

1,1,

VS18 Record Rules: Why They Exist and How Domains Work

Objectives, key points, and lab target

Objectives

- Explain record rules as row-level filters.
- Write domains that match business ownership.
- Understand global and group-specific rule behavior.

Key points

- Rules are domains on records.
- Model access must pass before record rules matter.
- Global rules combine with group rules.
- Rules can be limited by operation.
- Domains should be readable and testable.

Lab target

Write a rule that lets teachers read only their assigned courses, then verify the list, form, export, and many2one search behavior.

VS18 Record Rules: Why They Exist and How Domains Work

Teaching notes

Record Rules: Why They Exist and How Domains Work is best understood as part of the contract between the ORM, the web client, and the business user. Record rules restrict which rows a user can see or change after model access rights have allowed the operation. A senior Odoo developer reads a view or security file as executable configuration: it changes what users can see, which records they can touch, and which operations become safe or dangerous in production.

The design decision starts before writing XML. Design rules from ownership and responsibility: own records, company records, assigned team records, or records in an approved state. Good modules keep layout, data defaults, and authorization close to the business process they serve, while avoiding clever view code that hides a weak model design.

Implementation should be small, explicit, and easy to inherit. Write domains using user, company, and field values, then assign rules to groups unless the rule truly applies globally. Prefer stable external ids, clear XPath anchors, model fields with meaningful states, and security definitions that can be reviewed by a functional lead.

Also remember

- Verification is not only clicking the happy path as an administrator. Test read, write, create, and unlink separately...
- In production, the cost of a poor record rules: why they exist and how domains work decision appears during upgrades,...

VS18 Record Rules: Why They Exist and How Domains Work

Example and pitfalls

Own-record rule

The rule limits teacher users to courses connected to their own user record.

```
<record id="rule_training_course_own_teacher" model="training_course">
  <field name="name">Teachers see their courses</field>
  <field name="model_id" ref="model_training_course">training_course</field>
  <field name="domain_force">[('teacher_user_id', 'user_id')]</field>
  <field name="groups" eval="[(4, ref('training.group_training_teacher'))]" />
</record>
```

Common mistakes

- Forgetting that read rules also affect many2one display names.
- Creating global rules when a group rule was intended.
- Using domains that depend on fields users cannot access.
- Testing only one operation.

VS19 Record Rules Student and Teacher Example

Objectives, key points, and lab target

Objectives

- Model a realistic student-teacher security matrix.
- Combine group access and record ownership.
- Validate cross-role behavior with fixtures.

Key points

- Students and teachers need different ownership fields.
- Managers often need a broader rule, not sudo everywhere.
- Related user fields simplify domains.
- Fixtures make rule testing repeatable.
- Rules affect UI and RPC access.

Lab target

Create three users and four enrollments, then prove each role sees the expected records through the UI and an XML-RPC read call.

VS19 Record Rules Student and Teacher Example

Teaching notes

Record Rules Student and Teacher Example is best understood as part of the contract between the ORM, the web client, and the business user. Education and training modules are excellent record-rule examples because students, teachers, and managers see overlapping but different data. A senior Odoo developer reads a view or security file as executable configuration: it changes what users can see, which records they can touch, and which operations become safe or dangerous in...

The design decision starts before writing XML. Give every role the smallest model access it needs, then use rules for ownership: students see their enrollments, teachers see their courses, managers see all. Good modules keep layout, data defaults, and authorization close to the business process they serve, while avoiding clever view code that hides a weak model design.

Implementation should be small, explicit, and easy to inherit. Use related fields such as `student_user_id` and `teacher_user_id` to make domains readable and avoid deep joins where possible. Prefer stable external ids, clear XPath anchors, model fields with meaningful states, and security definitions that can be reviewed by a functional lead.

Also remember

- Verification is not only clicking the happy path as an administrator. Create fixture users for each role and run...
- In production, the cost of a poor record rules student and teacher example decision appears during upgrades, audits,...

VS19 Record Rules Student and Teacher Example

Example and pitfalls

Student and teacher domains

The student rule follows direct ownership while the teacher rule follows the course relationship.

```
<record id="rule_enrollment_student_self" model="ir.rule">
  <field name="name">Students see own enrollments</field>
  <field name="model_id" ref="model_training_enrollment"/>
  <field name="domain_force">[('student_user_id', 'self')]</field>
  <field name="groups" eval="[(4, ref('training_group_id'))]"</field>
</record>

<record id="rule_enrollment_teacher_courses" model="ir.rule">
  <field name="name">Teachers see course enrollments</field>
  <field name="model_id" ref="model_training_enrollment"/>
# ...
```

Common mistakes

- Letting students read the course roster through an unrestricted related model.
- Using sudo in controllers instead of fixing the rule design.
- Forgetting portal or external users if the module exposes them.
- Assuming a hidden menu blocks RPC reads.

VS20 Security Inheritance and Groups on Fields, Views, and Actions

Objectives, key points, and lab target

Objectives

- Apply groups attributes to UI elements.
- Understand how inherited security data changes effective permissions.
- Avoid confusing UI restrictions with real authorization.

Key points

- groups attributes hide UI elements from users outside the group.
- Menus and actions can be group-restricted.
- Field-level groups can protect field visibility.
- Server-side checks remain necessary.
- Inherited modules can widen or narrow effective access.

Lab target

Hide a margin field and finance action from normal users, then attempt to read the field by RPC to confirm whether server security is sufficient.

VS20 Security Inheritance and Groups on Fields, Views, and Actions

Teaching notes

Security Inheritance and Groups on Fields, Views, and Actions is best understood as part of the contract between the ORM, the web client, and the business user. Security inheritance happens when modules add groups, access rows, rules, or UI restrictions on top of existing behavior. A senior Odoo developer reads a view or security file as executable configuration: it changes what users can see, which records they can touch, and which operations become safe or dangerous in...

The design decision starts before writing XML. Use groups on fields, buttons, menus, views, and actions to reduce clutter for users who cannot use those features anyway. Good modules keep layout, data defaults, and authorization close to the business process they serve, while avoiding clever view code that hides a weak model design.

Implementation should be small, explicit, and easy to inherit. Keep actual data protection in access rights, record rules, and business methods; groups on views are a presentation and workflow control. Prefer stable external ids, clear XPath anchors, model fields with meaningful states, and security definitions that can be reviewed by a functional lead.

Also remember

- Verification is not only clicking the happy path as an administrator. Test inherited modules in the full dependency...
- In production, the cost of a poor security inheritance and groups on fields, views, and actions decision appears...

VS20 Security Inheritance and Groups on Fields, Views, and Actions

Example and pitfalls

Group-restricted margin field and action

Managers see finance information in the view and can access the reporting action, but model security must still protect the data.

```
<field name="margin_percent" groups="training.group_training_manager">
  <record id="training_finance_action" model="ir.actions_server"
    <field name="name">Course Margins</field>
    <field name="res_model">training.course</field>
    <field name="view_mode">pivot,graph</field>
    <field name="groups_id" eval="[(4, ref('training.group_training_manager'))]" />
  </record>
```

Common mistakes

- Believing hidden fields are secure if the model allows read.
- Adding group attributes in many inherited views without a central role design.
- Forgetting server checks on object buttons.
- Not reviewing implied groups when a module is installed.

5 VS21 – VS25

Views, Security & QWeb · teaching block 5

VS21 Odoo 19 Privilege Model

Objectives, key points, and lab target

Objectives

- Understand the direction of Odoo 19 privilege tightening.
- Identify where custom modules depend on broad administrative behavior.
- Plan privilege reviews during upgrades.

Key points

- Expect stronger separation between business and technical privileges.
- Audit sudo calls and administrative group checks.
- Do not assume old Settings access remains enough.
- Test server actions and scheduled jobs during upgrade.
- Treat privilege changes as security changes, not only migration work.

Lab target

Review one custom module and list every sudo call, then classify each as required, removable, or needing a preceding group check.

VS21 Odoo 19 Privilege Model

Teaching notes

Odoo 19 Privilege Model is best understood as part of the contract between the ORM, the web client, and the business user. Odoo 19 continues the trend of making privileges more explicit, especially around administrative operations, technical settings, and cross-app access. A senior Odoo developer reads a view or security file as executable configuration: it changes what users can see, which records they can touch, and which operations become safe or dangerous in production.

The design decision starts before writing XML. Audit custom modules for places where code assumes that a user with one application role can manage technical records or operate outside their business scope. Good modules keep layout, data defaults, and authorization close to the business process they serve, while avoiding clever view code that hides a weak model design.

Implementation should be small, explicit, and easy to inherit. Keep methods explicit about sudo usage, group checks, and record ownership so a framework privilege change does not silently break or overexpose behavior. Prefer stable external ids, clear XPath anchors, model fields with meaningful states, and security definitions that can be reviewed by a functional lead.

Also remember

- Verification is not only clicking the happy path as an administrator. During verification, run upgrade tests with real...
- In production, the cost of a poor odoo 19 privilege model decision appears during upgrades, audits, and support...

VS21 Odoo 19 Privilege Model

Example and pitfalls

Explicit group check around sensitive action

The method checks the business role before using sudo for the narrow internal operation.

```
def action_recompute_certificates(self):  
    if not self.env.user.has_group("training.group_train_manager"):  
        raise AccessError("Only training managers can recompute certificates")  
    self.sudo()._recompute_certificate_numbers()  
    return True
```

Common mistakes

- Using sudo to make upgrade errors disappear.
- Assuming a technical user interface group equals business approval.
- Leaving server actions callable by broad groups.
- Skipping tests for scheduled and automated actions.

VS22 Odoo 18 to 19 Migration Notes for Views and Security

Objectives, key points, and lab target

Objectives

- Plan view modifier migration work.
- Review deprecated XML patterns.
- Regression-test security after framework upgrades.

Key points

- Inventory before editing.
- Migrate modifiers deliberately.
- Review groups, access rights, and record rules together.
- Test reports and server actions, not only menus.
- Keep migration commits focused.

Lab target

Build a migration checklist for one module, convert two legacy modifiers, and document one security rule that needs a functional owner review.

VS22 Odoo 18 to 19 Migration Notes for Views and Security

Teaching notes

Odoo 18 to 19 Migration Notes for Views and Security is best understood as part of the contract between the ORM, the web client, and the business user. Migration from Odoo 18 to 19 should be treated as a behavioral review, not a search-and-replace exercise. A senior Odoo developer reads a view or security file as executable configuration: it changes what users can see, which records they can touch, and which operations become safe or dangerous in production.

The design decision starts before writing XML. Inventory view modifiers, JavaScript widgets, access files, record rules, automated actions, and reports before changing code. Good modules keep layout, data defaults, and authorization close to the business process they serve, while avoiding clever view code that hides a weak model design.

Implementation should be small, explicit, and easy to inherit. Update XML syntax where required, remove obsolete patterns, and prefer direct modifier expressions and clear inherited views over compatibility clutter. Prefer stable external ids, clear XPath anchors, model fields with meaningful states, and security definitions that can be reviewed by a functional lead.

Also remember

- Verification is not only clicking the happy path as an administrator. Run upgrade tests from a fresh restored...
- In production, the cost of a poor odoo 18 to 19 migration notes for views and security decision appears during...

VS22 Odoo 18 to 19 Migration Notes for Views and Security

Example and pitfalls

Migration checklist fragment

The first pass locates legacy modifiers, privilege-sensitive code, and security XML that deserve manual review.

Suggested review commands

```
rg "attrs=|states=" custom_addons/training
rg "sudo\(|has_group\(" custom_addons/training
rg "ir.rule|ir.model.access|groups=" custom_addons/training
```

Common mistakes

- Migrating syntax without comparing behavior.
- Leaving obsolete inherited views that no longer match upstream anchors.
- Testing only with admin users.
- Combining migration with unrelated refactors.

VS23 XML Data, CSV Data, and Choosing Between Them

Objectives, key points, and lab target

Objectives

- Choose XML or CSV based on data shape.
- Load reference data idempotently.
- Understand noupdate implications.

Key points

- XML is better for relational configuration records.
- CSV is better for large flat datasets.
- noupdate affects later module updates.
- External ids make data repeatable.
- Manifest order controls references.

Lab target

Create the same three category records in XML and CSV, then decide which version should ship and explain the upgrade behavior.

VS23 XML Data, CSV Data, and Choosing Between Them

Teaching notes

XML Data, CSV Data, and Choosing Between Them is best understood as part of the contract between the ORM, the web client, and the business user. Odoo modules use data files to install configuration, security, records, actions, and sometimes initial business data. A senior Odoo developer reads a view or security file as executable configuration: it changes what users can see, which records they can touch, and which operations become safe or dangerous in production.

The design decision starts before writing XML. Use XML when records need references, nested structure, eval expressions, noupdate control, or readable grouping; use CSV for flat bulk rows. Good modules keep layout, data defaults, and authorization close to the business process they serve, while avoiding clever view code that hides a weak model design.

Implementation should be small, explicit, and easy to inherit. Keep reference data stable with external ids and be careful with noupdate because it protects user-edited records from future module updates. Prefer stable external ids, clear XPath anchors, model fields with meaningful states, and security definitions that can be reviewed by a functional lead.

Also remember

- Verification is not only clicking the happy path as an administrator. Test install, update, uninstall where relevant,...
- In production, the cost of a poor xml data, csv data, and choosing between them decision appears during upgrades,...

VS23 XML Data, CSV Data, and Choosing Between Them

Example and pitfalls

Reference category XML

noupdate is appropriate when administrators may rename categories after installation.

```
<odoo nouupdate="1">
  <record id="course_category_functional" model="training.course.category">
    <field name="name">Functional</field>
    <field name="sequence">10</field>
  </record>
  <record id="course_category_technical" model="training.course.category">
    <field name="name">Technical</field>
    <field name="sequence">20</field>
  </record>
</odoo>
```

Common mistakes

- Using CSV for records that need complex references or eval.
- Putting user-editable configuration outside nouupdate without a reason.
- Referencing records before their files load.
- Duplicating records by removing external ids.

VS24 View Inheritance Overview and XPath Positions

Objectives, key points, and lab target

Objectives

- Use inherited views instead of copying base views.
- Select robust XPath anchors.
- Apply position attributes safely.

Key points

- inherit_id points to the parent view.
- XPath anchors should be stable and specific.
- position='attributes' changes attributes only.
- Avoid copying full upstream views.
- One inherited view should have one clear purpose.

Lab target

Inherit a form twice from two small modules, then inspect how priority and XPath anchors affect the final architecture.

VS24 View Inheritance Overview and XPath Positions

Teaching notes

View Inheritance Overview and XPath Positions is best understood as part of the contract between the ORM, the web client, and the business user. View inheritance lets modules extend or change existing screens without replacing the upstream view entirely. A senior Odoo developer reads a view or security file as executable configuration: it changes what users can see, which records they can touch, and which operations become safe or dangerous in production.

The design decision starts before writing XML. Choose XPath anchors that express intent and are likely to survive upgrades, such as named fields or groups with stable names. Good modules keep layout, data defaults, and authorization close to the business process they serve, while avoiding clever view code that hides a weak model design.

Implementation should be small, explicit, and easy to inherit. Use positions before, after, inside, replace, attributes, and move carefully, keeping each inherited view focused on one business change. Prefer stable external ids, clear XPath anchors, model fields with meaningful states, and security definitions that can be reviewed by a functional lead.

Also remember

- Verification is not only clicking the happy path as an administrator. Verify the final combined architecture in...
- In production, the cost of a poor view inheritance overview and xpath positions decision appears during upgrades,...

VS24 View Inheritance Overview and XPath Positions

Example and pitfalls

Add a field after teacher

The extension adds one field and changes one attribute without copying the whole parent form.

```
<record id="training_course_form_inherit_capacity" mod
  <field name="name">training.course.form.inherit.ca
  <field name="model">training.course</field>
  <field name="inherit_id" ref="training.training_co
  <field name="arch" type="xml">
    <xpath expr="//field[@name='teacher_id']" position="after">
      <field name="seat_count"/>
    </xpath>
    <xpath expr="//field[@name='price_total']" position="attributes">
      <attribute name="groups">training.group_training_manager</attribute>
  # ...
```

Common mistakes

- Using positional XPaths like `//group[2]` that break after small upstream changes.
- Replacing large blocks when attributes would be enough.
- Combining unrelated changes in one inherited view.
- Ignoring priority when multiple modules inherit the same target.

VS25 ir.sequence for Business Numbers

Objectives, key points, and lab target

Objectives

- Create reliable sequence records.
- Assign sequence values in create.
- Handle company-specific numbering when needed.

Key points

- Use code as the stable lookup key.
- Assign numbers server-side in create.
- Do not recompute sequence values on write.
- Date ranges and companies change numbering behavior.
- Legal sequences may require stricter gap handling.

Lab target

Add a certificate number sequence, create records through UI and import, and confirm every saved record receives one unique number.

VS25 ir.sequence for Business Numbers

Teaching notes

ir.sequence for Business Numbers is best understood as part of the contract between the ORM, the web client, and the business user. Business users expect order numbers, certificate numbers, and ticket references to be unique, readable, and stable. A senior Odoo developer reads a view or security file as executable configuration: it changes what users can see, which records they can touch, and which operations become safe or dangerous in production. The design decision starts before writing XML. Decide whether the number is legal, operational, or cosmetic because that determines gaps, prefixes, date ranges, and company separation. Good modules keep layout, data defaults, and authorization close to the business process they serve, while avoiding clever view code that hides a weak model design.

Implementation should be small, explicit, and easy to inherit. Define an ir.sequence record with code, prefix, padding, and implementation, then call `next_by_code` in create only when the placeholder value is still new. Prefer stable external ids, clear XPath anchors, model fields with meaningful states, and security definitions that can be reviewed by a functional lead.

Also remember

- Verification is not only clicking the happy path as an administrator. Test imports, duplicate creation, multi-company...
- In production, the cost of a poor ir.sequence for business numbers decision appears during upgrades, audits, and...

VS25 ir.sequence for Business Numbers

Example and pitfalls

Certificate sequence

The XML defines the sequence and create assigns it once when the record is created.

```
<record id="seq_training_certificate" model="ir.sequence">
  <field name="name">Training Certificate</field>
  <field name="code">training.certificate</field>
  <field name="prefix">CERT/%(year)s</field>
  <field name="padding">5</field>
</record>
```

```
@api.model_create_multi
def create(self, vals_list):
    for vals in vals_list:
        # ...
```

Common mistakes

- Calling `next_by_code` in `default_get` and consuming numbers for abandoned forms.
- Changing sequence prefixes without migration communication.
- Using one sequence when companies require separate ranges.
- Allowing users to edit legal sequence numbers casually.

6 VS26 – VS28

Views, Security & QWeb · teaching block 6

VS26 QWeb Intro and Core Directives

Objectives, key points, and lab target

Objectives

- Explain how QWeb renders XML templates.
- Use t-call, t-set, t-esc, and t-att safely.
- Separate template logic from business logic.

Key points

- QWeb templates are XML and must be valid XML.
- t-esc escapes output and is safer for user data.
- t-call composes reusable templates.
- t-att builds dynamic attributes.
- Business calculations belong outside templates.

Lab target

Create a QWeb template that renders a certificate title, student name, teacher name, and escaped note field.

VS26 QWeb Intro and Core Directives

Teaching notes

QWeb Intro and Core Directives is best understood as part of the contract between the ORM, the web client, and the business user. QWeb is Odoo's XML templating engine for reports, website fragments, mail snippets, and many kanban templates. A senior Odoo developer reads a view or security file as executable configuration: it changes what users can see, which records they can touch, and which operations become safe or dangerous in production.

The design decision starts before writing XML. Keep templates focused on presentation and put calculations, permissions, and data preparation in models or report helpers. Good modules keep layout, data defaults, and authorization close to the business process they serve, while avoiding clever view code that hides a weak model design.

Implementation should be small, explicit, and easy to inherit. Use t-call for composition, t-set for local values, t-esc for escaped output, and t-att or t-attf for attributes that come from safe values. Prefer stable external ids, clear XPath anchors, model fields with meaningful states, and security definitions that can be reviewed by a functional lead.

Also remember

- Verification is not only clicking the happy path as an administrator. Render reports with representative records and...
- In production, the cost of a poor qweb intro and core directives decision appears during upgrades, audits, and support...

VS26 QWeb Intro and Core Directives

Example and pitfalls

Reusable certificate block

The template escapes record values and uses a dynamic class for styling without doing heavy calculation in QWeb.

```
<template id="training_certificate_block">
  <div class="certificate">
    <h2><t t-esc="doc.name"/></h2>
    <p>Issued to <span t-esc="doc.student_id.name"/></p>
    <p t-att-class="'grade grade-%s' % doc.grade_code">
      Score: <span t-esc="doc.score"/>
    </p>
  </div>
</template>
```

Common mistakes

- Using t-raw for user-provided data.
- Putting complex calculations in template expressions.
- Forgetting XML escaping rules.
- Copying templates instead of using t-call.

VS27 QWeb Conditionals, Loops, and Template Inheritance

Objectives, key points, and lab target

Objectives

- Use t-if and t-foreach for report structures.
- Inherit QWeb templates with XPath.
- Keep repeated report sections maintainable.

Key points

- t-if controls conditional rendering.
- t-foreach repeats over recordsets or lists.
- Template inheritance uses XPath like view inheritance.
- Prepare complex data before rendering.
- Test empty and long datasets.

Lab target

Build a roster report that shows a table for enrolled students and a clear message when no students are enrolled.

VS27 QWeb Conditionals, Loops, and Template Inheritance

Teaching notes

QWeb Conditionals, Loops, and Template Inheritance is best understood as part of the contract between the ORM, the web client, and the business user. Most useful QWeb reports need conditional sections and repeated lines, but that does not mean the template should become a program. A senior Odoo developer reads a view or security file as executable configuration: it changes what users can see, which records they can touch, and which operations become safe or dangerous in...

The design decision starts before writing XML. Prepare clean recordsets in Python and let QWeb decide only whether to show sections and how to repeat display rows. Good modules keep layout, data defaults, and authorization close to the business process they serve, while avoiding clever view code that hides a weak model design.

Implementation should be small, explicit, and easy to inherit. Use t-if, t-elif, t-else, t-foreach, t-as, and inherited templates with XPath to extend report layouts without copying them. Prefer stable external ids, clear XPath anchors, model fields with meaningful states, and security definitions that can be reviewed by a functional lead.

Also remember

- Verification is not only clicking the happy path as an administrator. Verify empty lists, one-row lists, and many-row...
- In production, the cost of a poor qweb conditionals, loops, and template inheritance decision appears during upgrades,...

VS27 QWeb Conditionals, Loops, and Template Inheritance

Example and pitfalls

Loop and inherit report template

The template handles both a roster and an empty state with simple QWeb directives.

```
<template id="report_course_roster">
  <t t-call="web.external_layout">
    <h2><span t-esc="doc.name"/></h2>
    <table>
      <tr t-foreach="doc.enrollment_ids" t-as="line">
        <td><span t-esc="line.student_id.name"/></td>
        <td><span t-esc="line.state"/></td>
      </tr>
    </table>
    <p t-if="not doc.enrollment_ids">No students enrolled.</p>
# ...
```

Common mistakes

- Looping over unsorted recordsets when report order matters.
- Using t-row to inject HTML without sanitizing.
- Copying external_layout instead of inheriting or calling it.
- Forgetting the empty state.

VS28 Paper Formats, Barcodes, Images, Server Actions, and Email Templates

Objectives, key points, and lab target

Objectives

- Configure report paper formats and media assets.
- Add barcode or QR output to PDFs.
- Connect server actions and email templates to document workflows.

Key points

- Paper format controls margins, orientation, and page size.
- Barcode and QR images can be rendered through report routes.
- Binary images need fallbacks and size control.
- Server actions should run with deliberate privileges.
- Email templates should be previewed with real records.

Lab target

Create a certificate PDF with a custom paper format, company logo, QR verification link, and an email template action.

VS28 Paper Formats, Barcodes, Images, Server Actions, and Email Templates

Teaching notes

Paper Formats, Barcodes, Images, Server Actions, and Email Templates is best understood as part of the contract between the ORM, the web client, and the business user. Printed documents are often where Odoo leaves the browser and becomes part of legal, warehouse, or customer-facing process. A senior Odoo developer reads a view or security file as executable configuration: it changes what users can see, which records they can touch, and which operations become safe or...

The design decision starts before writing XML. Define paper format before fine-tuning CSS, because margins, orientation, and DPI change how every report element behaves. Good modules keep layout, data defaults, and authorization close to the business process they serve, while avoiding clever view code that hides a weak model design.

Implementation should be small, explicit, and easy to inherit. Use report barcode routes for QR codes, serve images from binary fields or attachments, and trigger emails through templates or server actions only after permissions are clear. Prefer stable external ids, clear XPath anchors, model fields with meaningful states, and security definitions that can be reviewed by a functional lead.

Also remember

- Verification is not only clicking the happy path as an administrator. Test PDFs with missing images, long names,...
- In production, the cost of a poor paper formats, barcodes, images, server actions, and email templates decision...

VS28 Paper Formats, Barcodes, Images, Server Actions, and Email Templates

Example and pitfalls

QR code and email send

The report renders a QR code from a verification URL and the model action sends the matching email template.

```


def action_send_certificate(self):
    self.ensure_one()
    template = self.env.ref("training.email_template_certificate")
    template.send_mail(self.id, force_send=True)
    return True
```

Common mistakes

- Hard-coding paper margins in CSS while ignoring paperformat.
- Embedding huge images that slow every PDF render.
- Running server actions with sudo without business checks.
- Sending emails without previewing language and company context.

VS28 Paper Formats, Barcodes, Images, Server Actions, and Email Templates

Extra patterns

Report production checklist

- Paper format selected
- Images tested when missing
- QR or barcode verified
- Email preview approved

Σ You should now be able to...

Section wrap-up

- VS01 Intro to Odoo Views and XML Data Loading
- VS02 List Views, Sorting, and Limits
- VS03 Editable Lists, Mass Editing, and the Handle Widget
- VS04 List Decorations, Field Decorations, Widths, and Hidden Labels
- VS05 List Totals, Averages, and Grouped Lists
- VS06 Search Views, Default Filters, and Dynamic Date Filters
- VS07 Search Panel and Groupby UX
- VS08 Form Customization with Groups, Tags, and Widgets
- Complete every lab before the next section
- Keep a command/error journal
- Prefer disposable databases for experiments
- Re-read pitfalls before production changes

Views

Security

XML

QWeb

NEXT STEP

Complete the section labs

Open the next presentation deck

Section 05 · Views & Security

